```
  def apply(): Behavior[String] =
    Behaviors
      .receivePartial[String] {
        ...
      }
      .receiveSignal {                                  ❶
        case (context, PostStop) =>                     ❷
          context.log.info("about to stop")
          Behaviors.same
      }
}
```

❶ Handles any signal

❷ Handles a specific signal

In this example, you can see how the protocol and signals are handled. Signals are handled by

```
def receiveSignal(ctx: TypedActorContext[T], msg: Signal): Behavior[T]
```

which has a very similar signature to

```
def receive(ctx: TypedActorContext[T], msg: T): Behavior[T]
```

which you've already seen for normal messages. The obvious difference is the type of `msg`, but it's also important to note that signals can only be sent from the underlying system to an actor. You can't send a signal to the actor because the signal doesn't belong to its type.

With the stop strategy, no more messages from the mailbox are processed because the next behavior is set to `Behavior.stopped`. After processing the `PostStop` signal, the mailbox and the supervisor are deleted together with the actor. You can set this strategy as follows: